Lab 4
Arithmetic-Logic Unit (ALU)

1. Introduction

The purpose of this lab is to design an arithmetic-logic unit (ALU) that operates on 4-bit binary numbers. The ALU will support four different operations: addition, subtraction, logic AND, and logic OR. The results will be displayed in binary format on the LEDs as well as in decimal format on the 7-Segment displays. You are **not** allowed to use any of the Quartus library modules unless for the modules that are explicitly allowed in the lab instructions.

1.1 Pre-Lab Assignment

An unsigned 4-bit binary number has values that range from 0 to 15. So, to display its decimal value, we need two 7-segments displays. But first we need to convert the 4-bit binary value to its binary-coded decimal (BCD) format as shown in the table.

In this lab you will need to implement a digital circuit that takes as an input the binary value, $B_3B_2B_1B_0$, and generate as an output its corresponding BCD value, $D_4D_3D_2D_1D_0$. For this pre-lab assignment, use five Karnaugh maps, based on the shown truth table, to find the minimized Boolean expression of each output bit ($D_4D_3D_2D_1D_0$) as a function of the input bits ($B_3B_2B_1B_0$).

Have the requirements of the prelab assignment ready to be checked at the beginning of the lab session.

Binary B_3 B_2 B_1 B_0	2-Digit Decimal	BCD D_4 D_3 D_2 D_1 D_0
0 0 0 0	00	0 0 0 0 0
0 0 0 1	01	0 0 0 0 1
0 0 1 0	02	0 0 0 1 0
0 0 1 1	03	0 0 0 1 1
0 1 0 0	04	0 0 1 0 0
0 1 0 1	05	0 0 1 0 1
0 1 1 0	06	0 0 1 1 0
0 1 1 1	07	0 0 1 1 1
1 0 0 0	08	0 1 0 0 0
1 0 0 1	09	0 1 0 0 1
1 0 1 0	10	1 0 0 0 0
1 0 1 1	11	1 0 0 0 1
1 1 0 0	12	1 0 0 1 0
1 1 0 1	13	1 0 0 1 1
1 1 1 0	14	1 0 1 0 0
1 1 1 1	15	1 0 1 0 1

2. Binary to BCD Decoder

We will start by implementing the decoder that takes a 4-bit binary number and generates its corresponding BCD 5-bit binary code. The block diagram of the decoder is shown in Figure 1. Carry out the following steps to implement and test the decoder:

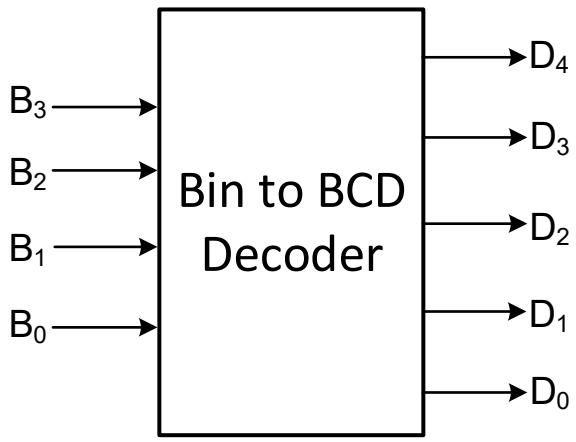

Figure 1: Binary to BCD Decoder

a) Create a new folder for a new project named **Bin2BCD** on your computer. Create the new project following the same steps in the *Quartus Prime Lite Reference* (in Canvas: Lab Resources). Set the project folder to the **Bin2BCD** folder and project name to **Bin2BCD**.
b) On Quartus Schematic, build the binary to BCD decoder circuits (four inputs and five outputs) based on your Boolean expressions from the pre-lab assignment.
c) Upload your design to the DE1-SoC board. For the 4-bit inputs, $B_3B_2B_1B_0$, use the first four switches: **SW[3], W[2]**, **SW[1]**, and **SW[0]**. For the 5-bit output, $D_4D_3D_2D_1D_0$, connect them to the first five LEDs: **LEDR[4]** , **LEDR[3]**, **LEDR[2]**, **LEDR[1]**, and **LEDR[0]**. Follow the same procedure outlined in the *Quartus Prime Lite Reference*.
d) Pack the **Bin2BCD** schematic as a logic block named **Bin2BCD.bsf** for later use. To do this, go back to your **Bin2BCD** schematic and on the **File** main menu, select **Create/Update → Create Symbol Files for Current File** and save the schematic as **Bin2BCD.bsf** file.

Assignment 1

Verify the correct behavior and output on the LEDs by changing the switches settings. Take pictures of your Board's switches and LEDs with at least four different settings for the switches and their corresponding outputs on the LEDs. Include them in your report.

Note: to avoid a large report file, make sure to reduce the resolution of the pictures by using a picture management tool that allow saving pictures for print purposes.

3. The 4-bit ALU Implementation

Now you need to implement the ALU components shown in the block diagram of Figure 2. Table 1 shows how the *ALUControl* input controls the ALU operations. The *ALUControl* input is used as an input to a 4-bit 4-to-1 MUX to select the operation. Also, bit 0 of *ALUControl* is used to select between the add and subtract operation to be performed by the 4-bit Add/Sub circuit, which you implemented in Lab 2. To simplify the ALU diagram, the AND and OR operations are shown using one logic gate for each. Quartus also allows connecting a 2-input AND/OR gate to two buses as inputs, as long as both buses have the same width (number of bits) and you are only using the corresponding bit from each bus as the inputs to the AND/OR operation. However, they are actually implemented as four 2-input AND gates for the AND operation and four 2-input OR gates for the OR operation.

Table 1: ALUControl Operations

ALUControl	Result
00	*SrcA* + *SrcB*
01	*SrcA* - *SrcB*
10	*SrcA* AND *SrcB*
11	*SrcA* OR *SrcB*

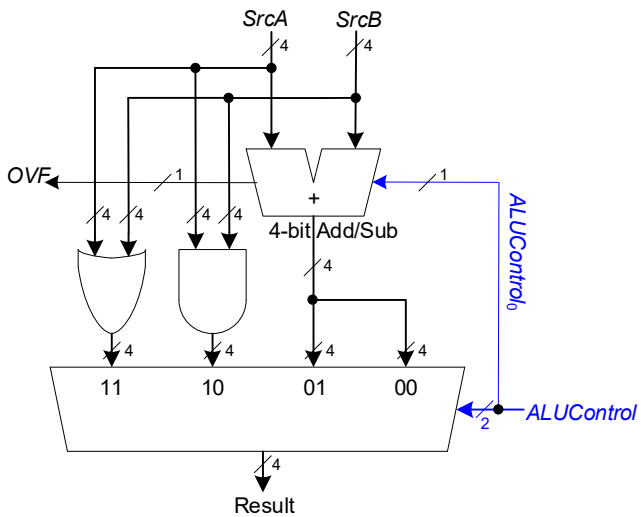

Figure 2: 4-bit ALU

a) Create a new folder for a new project named **Four_bit_ALU** and, from your previous projects folders, copy the **half_adder.bsf**, **half_adder.bdf**, **full_adder.bsf** , **full_adder.bdf**, **four_bit_add_sub.bsf** and **four_bit_add_sub.bdf** files to this new project folder.

b) On Quartus Schematic, build the above ALU circuits in the new project **Four_bit_ALU**. The circuit has two 4-bit data inputs and 2-bit control input. The outputs are the 4-bit result and 1-bit overflow bit. The ALUControl 2-bit input is the select line of a 4-bit 4-to-1 MUX.
   - No need to implement this Mux from scratch, instead follow the steps in sub-section **3.1** below to add a multiplexer module from the Quartus library.
   - Also, in different places in your design you will need to connect individual wires to buses (group of wires representing one thing such as the 4-bit Result bus of the Mux). Check sub-section **3.2** below to learn how to do that.

c) Assign the first 4 switches on the DE1-SoC board to input ***SrcA*** and the following 4 switches to input ***SrcB***. Use the last two switch on the board, **SW[8]** and **SW[9]**, for the ***ALUControl*** input.

d) Assign the 4-bit result output to the first 4 LEDs and the overflow bit to the **LEDR[9]** on the board.

e) Validate your design on the DE1-SoC board with different inputs to test all scenarios.

f) Pack the **Four_bit_ALU** schematic as a logic block named **Four_bit_ALU.bsf** for later use. To do this, go back to your **Four_bit_ALU** schematic and on the **File** main menu, select **Create/Update → Create Symbol Files for Current File** and save the schematic as **Four_bit_ALU.bsf** file.

Assignment 2

Verify the correct behavior of your ALU design by changing its data and control inputs. Take pictures of your Board's switches and LEDs with at least six different settings of the switches and their corresponding outputs on the LEDs. Make sure to include add and subtract operations with and without overflow. Take screen captures of your design and include them in your report.

3.1. Adding a Mux from the library

Quartus has a library of modules that you can insert in your design after configuring their parameters based on your design needs. In this lab, to add a 4-bit 4-to-1 MUX module, do the following:

a) To add the LPM_MUX module, select the component from the IP Catalog library (available from the right pane): *Library → Basic Functions → Miscellaneous → LPM_MUX.*

b) As shown in Figure 3 below, enter *Mux4bit4to1* for its IP variation file name.

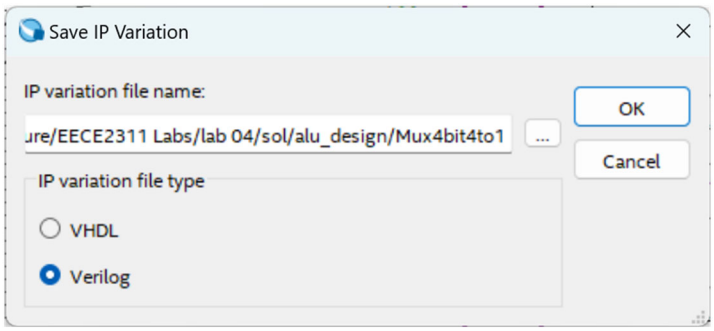

Figure 3: Name the Component

c) Set the number of inputs to 4 and the data width to 4 bits and Click *Next*.

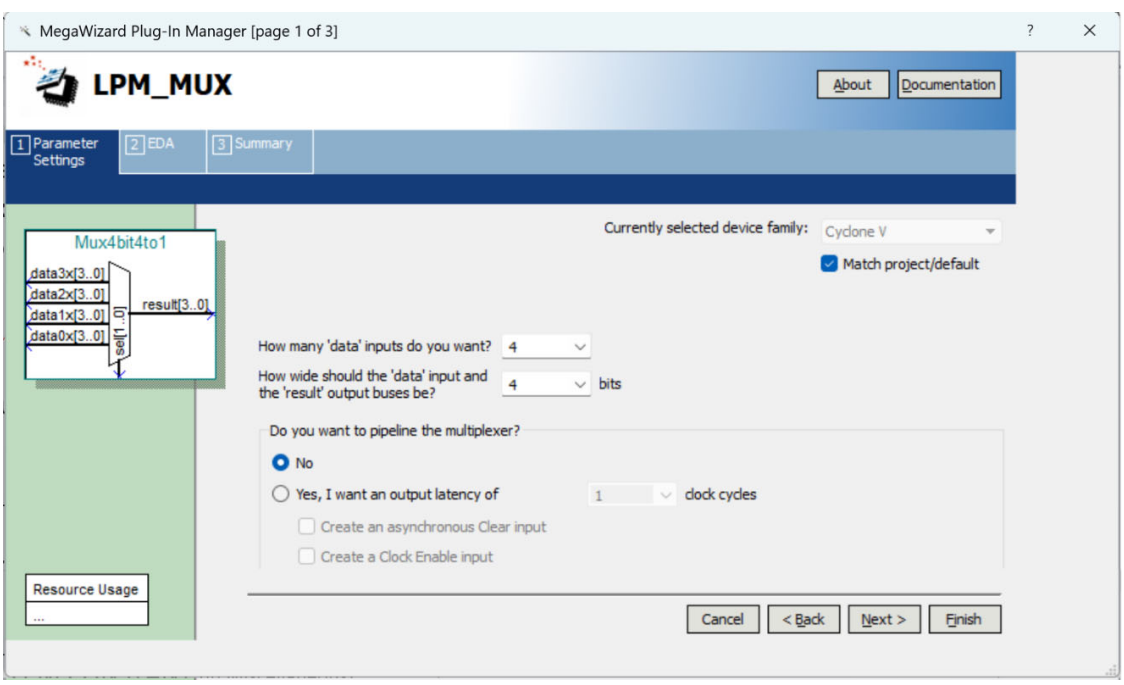

Figure 4: 4-bit 4-to-1 Mux

d) Leave the default setting on the *EDA* window for this and all other components. Click *Next*.

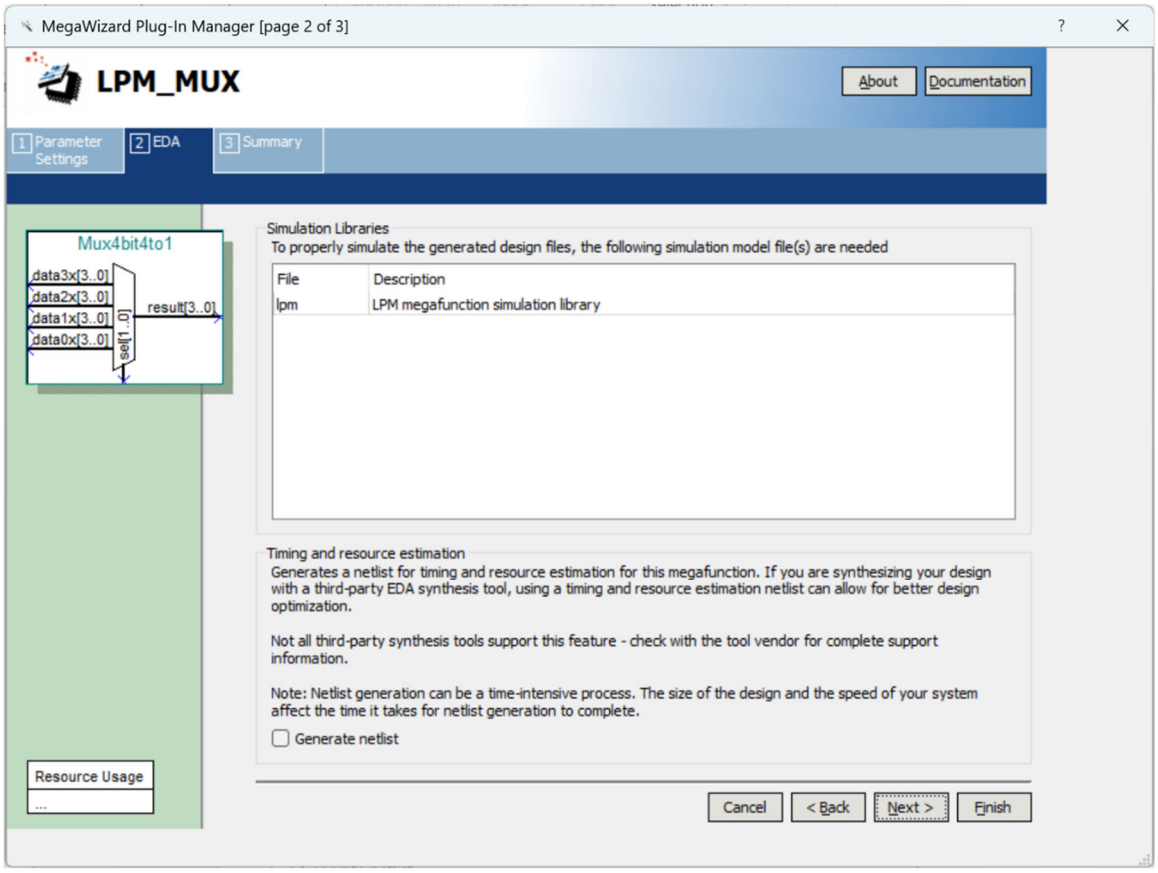

Figure 5: Leave Default Settings.

e) On the *Summary* window below, make sure to **check everything** as shown below in Figure 6 to generate the files needed for this component. Click *Finish*.

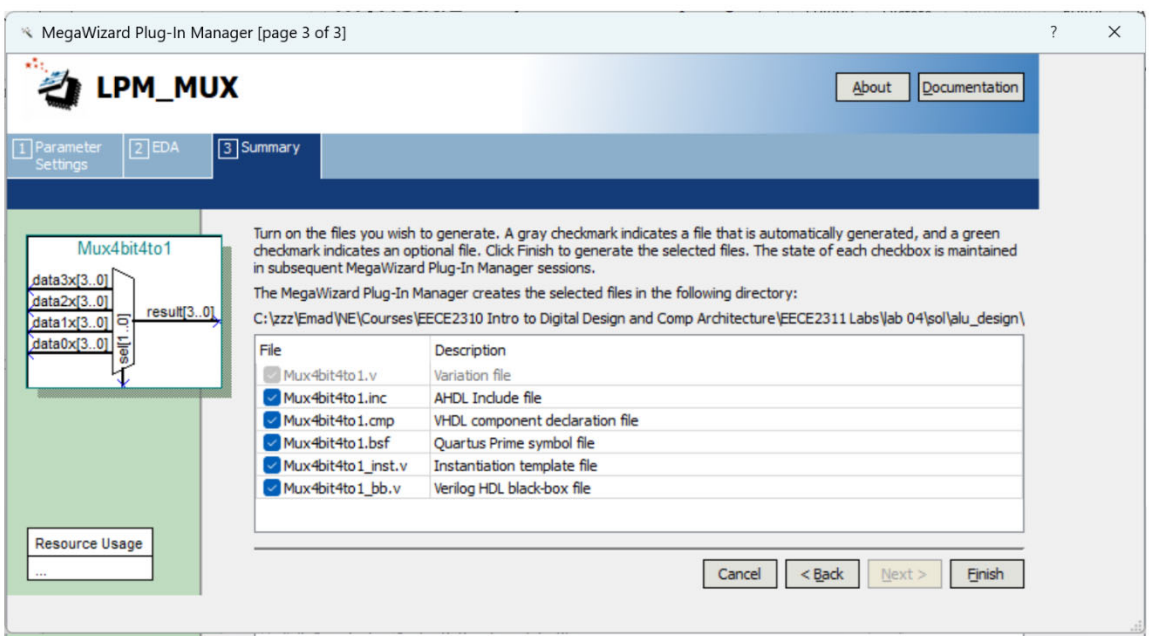

Figure 6: Check Everything to Generate Correct Files

f) Click **Yes** on the *IP Files* window to add the required files into your project.

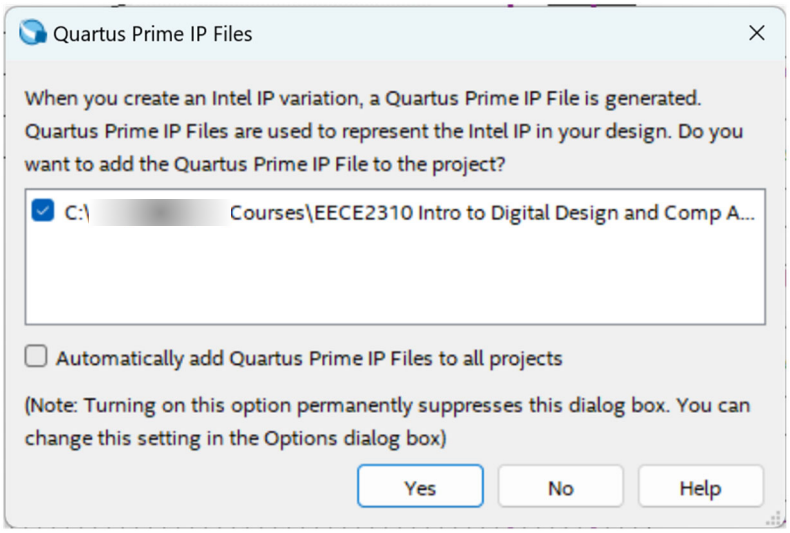

Figure 7: Click Yes to add the Required Files into Your Project

g) The new *Mux4bit4to1* module will be available from the **Symbol Tool library** (on the top of the dotted screen)**.** You can add instances (objects) from this Mux to your design as needed.

3.2. Connecting Wires to a Bus

To connect logic components, you can use the Bus Tool for data buses (multiple lines in one connection) or the Node Tool (single line connection) as needed. For example, as shown in Figure 8, to connect the 4-bit Mux result bus in the design to four individual wires representing the outputs, which will be eventually mapped to LEDs, do the following:

1. Extend the Mux result output bus and name this extension R[3..0].
2. Extend each of the four individual wires and name them R[0], R[1], R[2], and R[3] as shown.
3. Simply connect these wires to the mux bus as shown, which will result in connecting the first output, R0, to result R[0] from the Mux and so on.

To name a connection, **Right click** on that connection wire/bus → Select **Properties** and fill in its **Name field**.

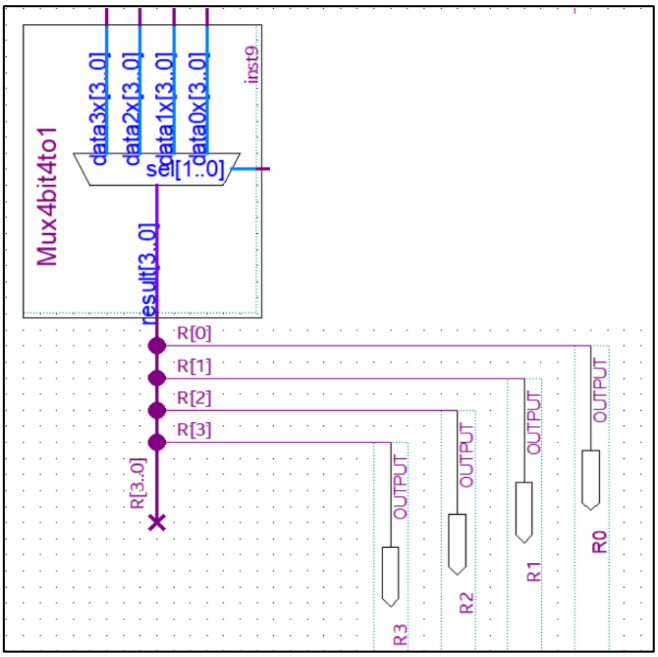

Figure 8: Connecting wires to a bus

4. The ALU Decimal Outputs on the 7-Segment Displays

In this section you will extend the **Four_bit_ALU** you implemented in the previous section to display the decimal value of its output on the 7-Segment Displays. You will need to utilize two of the displays: **HEX0** and **HEX1** using the decoder implemented in Lab 3 and shown in Figure 9. The decimal value depends on the operation performed by the ALU as explained in Table 2.

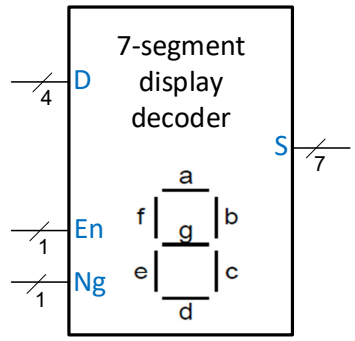

Figure 9: 7-Segment Decoder

ALU Operation	Expected Decimal Output on HEX1 HEX0
ADD or SUB (with no overflow)	-8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7
ADD or SUB (with overflow)	88
AND or OR	00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15

Table 2: 7-segments expected outputs

Based on these expected outputs, Table 3 below elaborates on how the ALU result must be displayed on HEX0 and HEX1 7-segments based on the values of the *ALUControl*, *OVF*, and *Result*$_3$ (the sign bit of the result). Note the following terminologies used in this table:

- $HEX0_D$ and $HEX1_D$ represent the 4-bit **D** inputs to be fed to the two 7-segment decoders (Figure 9).
- $HEX1_{Ng}$ is the 1-bit **Ng** input to the HEX1 7-segment decoder.
- $HEX1_{En}$ is the 1-bit **En** input to the HEX1 7-segment decoder.
- The 1-bit **Ng** input to the HEX0 7-segment decoder is always OFF.
- The 1-bit **En** input to the HEX0 7-segment decoder is always ON.
- $Result_{3-0}$ (where $Result_3$ is the most significant bit) are the 4-bit input to the *Binary to BCD Decoder* (Figure 1), while D_4 and D_{3-0} are its outputs.
- "x" in an output column means don't care while it means all possible values if is in an input column.

Circuits Inputs			Circuits Outputs			
$ALUControl_{1-0}$	OVF	$Result_3$	$HEX1_{En}$	$HEX1_{Ng}$	$HEX1_D$	$HEX0_D$
0 0	0	0	0	0	x	$Result_{3-0}$
0 0	0	1	1	1	x	2's complement($Result_{3-0}$)
0 0	1	x	1	0	8	8
0 1	0	0	0	0	x	$Result_{3-0}$
0 1	0	1	1	1	x	2's complement($Result_{3-0}$)
0 1	1	x	1	0	8	8
1 0	x	x	1	0	D_4	D_{3-0}
1 1	x	x	1	0	D_4	D_{3-0}

Table 3: Truth tables of the 7-segments control circuits

For example, **$HEX1_D$** is **x** in the first row because HEX1 is disabled (**$HEX1_{En}$** = 0) in that row, which means empty HEX1 will be shown regardless of the **$HEX1_D$** value. **$HEX1_D$** is also **x** in the second row because the **NG** input to HEX1 is enabled (**$HEX1_{Ng}$** = 1), which means HEX1 will display the negative sign regardless of the **$HEX1_D$** value. In the third row, **x** for the **$Result_3$** input indicates that for any value of the sign bit **$Result_3$** the output will be the same (i.e., 88) because the overflow flag (**OVF**) is 1.

You will need to build a number of combinational circuits to generate the outputs listed on Table 3. **$HEX1_{En}$** and **$HEX1_{Ng}$** are straightforward to implement with the help of two k-maps. As for **$HEX1_D$** and **$HEX0_D$** they can be replaced with the select lines, **$HEX1_S$** and **$HEX0_S$**, of the multiplexers shown in Figure 10. This will result in the new Table 4 as a modified version of Table 3. As shown, these select lines are represented by the three outputs **$HEX1_S$**, **$HEX0_S$[1]**, and **$HEX0_S$[0]** (as **$HEX0_S$** has two bits). In the table every input and output are associated with a one-letter variable (i.e., A, B, … etc.) to simplicity your K-maps. So, you will need five 4-input Karnaugh maps to implement the circuits of the five outputs in Table 4. To prepare the inputs of the multiplexers in Figure 10, you will need to feed the ALU result to a Binary to BCD Decoder and to another circuit to find its 2's complement.

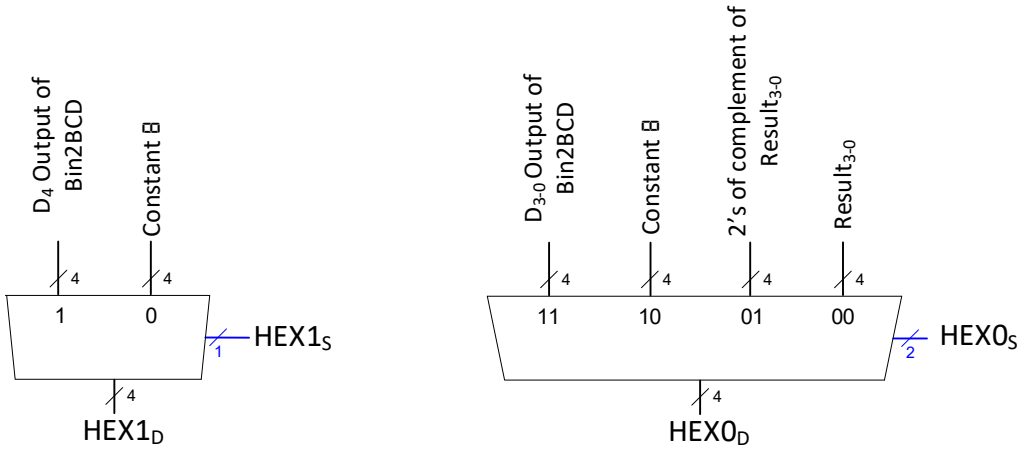

Figure 10: Multiplexers to generate $HEX1_D$ and $HEX0_D$ of Table 3

Circuits Inputs				Circuits Outputs				
$ALUControl_1$	**$ALUControl_0$**	**OVF**	**$Result_3$**	**$HEX1_{En}$**	**$HEX1_{Ng}$**	**$HEX1_S$**	**$HEX0_S$[1]**	**$HEX0_S$[0]**
A	**B**	**C**	**D**	**V**	**W**	**X**	**Y**	**Z**
0	0	0	0	0	0	x	0	0
0	0	0	1	1	1	x	0	1
0	0	1	x	1	0	0	1	0
0	1	0	0	0	0	x	0	0
0	1	0	1	1	1	x	0	1
0	1	1	x	1	0	0	1	0
1	0	x	x	1	0	1	1	1
1	1	x	x	1	0	1	1	1

Table 4: Modified truth tables of the 7-segments control circuits

The following are the steps you need to follow to implement a circuit that displays the output of the ALU on the two 7-segment displays based on the above requirements.

a) Create a new folder for a new project named **Four_bit_ALU_7seg** and, from your previous projects folders, copy the following files to this new project folder:
   - **half_adder.bsf**, **half_adder.bdf**
   - **full_adder.bsf**, **full_adder.bdf**
   - **four_bit_add_sub.bsf**, **four_bit_add_sub.bdf**
   - **display7_en.bsf**, **display7_en.bdf**
   - **Bin2BCD.bsf**, **Bin2BCD.bdf**
   - **Four_bit_ALU.bsf**, **Four_bit_ALU.bdf**
   - **Mux4bit4to1.bsf, Mux4bit4to1.v**

b) On Quartus Schematic, add the following:
   - One instant of the **Four_bit_ALU** circuit.
   - One instant of the **Bin2BCD** circuit.
   - Two instants of the **display7_en** circuit.
   - One instant of the **four_bit_add_sub** circuit.
   - One instant of the **Mux4bit4to1** circuit (needed for the right Mux on Figure 10).

c) As for the left Mux on Figure 10, follow the steps in subsection 3.1 to generate a **Mux4bit2to1** module. The difference here is that this Mux has only 2 inputs.

d) Figure 11 below shows how you need to connect the above modules together, where:
   - The combinational circuits block must be replaced with the five digital circuits resulted from the Karnaugh maps based on the truth tables in Table 4.
   - The 2's complement block can be implemented using the **four_bit_add_sub** circuit, where you need to subtract **Result$_{3\text{-}0}$** from value 0.
   - **VCC** and **GND** are used to apply constant logic 1 and 0 respectively. They are available in the Symbol Tool (type them in the search field). You need to use them, as shown in the figure, to always enable HEX0 and disable its **Ng** input. You can also use them to apply **Constant 8** and **`000:D4`** inputs to the Muxes.

     Note: input **`000:D4`** represents three zeros and bit number 4 of the **Bin2BCD** decoder output.
   - Pay attention to the order of the inputs to the top Mux in the figure. For example, D[3..0] is connected to input $(11)_2$ of that Mux.
   - Finally, you need to show the **Result** on **four LEDs** as in future lab we will need both the binary and decimal outputs of the ALU.

e) Pack the **Four_bit_ALU_7seg** schematic as a logic block named **Four_bit_ALU_7seg.bsf** for later use. To do this, go back to your **Four_bit_ALU_7seg** schematic and on the **File** main menu, select **Create/Update → Create Symbol Files for Current File** and save the schematic as **Four_bit_ALU_7seg.bsf** file.

Assignment 3

Verify the correct behavior of your **Four_bit_ALU_7seg** design with at least eight different scenarios of its data and control inputs to test all implemented operations with different result values (e.g., result +ve, -ve, and with overflow). Take a picture of your Board's switches, LEDs, and 7-segment displays for the above scenarios. Take screen captures of your design and include them in your report.

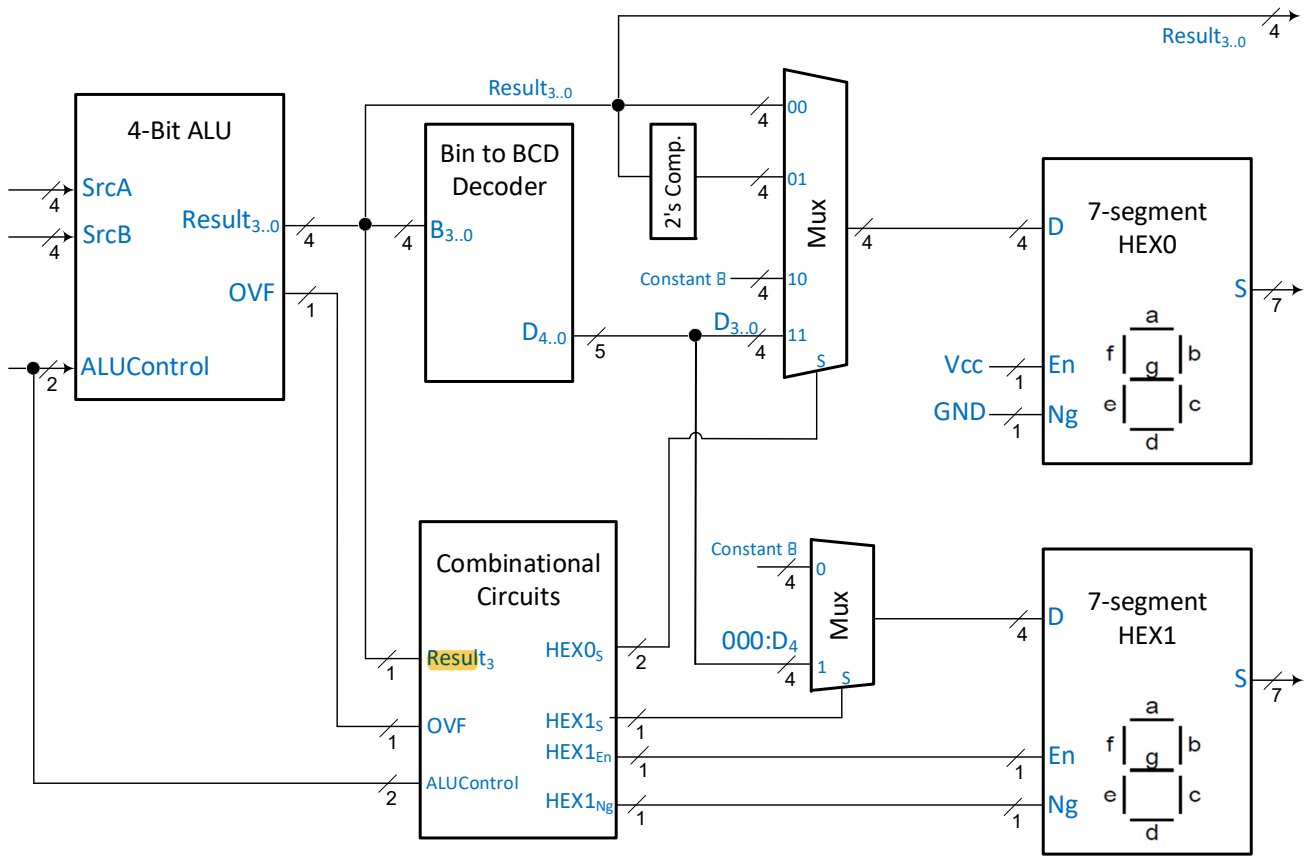

Figure 11: Block diagram of the 4-bit ALU and 7 Segments

5. Lab Submission Instructions

- The lab is intended to be implemented by each **individual** student. It is also allowed for a **maximum of two students** to work in the lab together. If you decide to work in a group of two students, **both students** must **check off** the working lab assignments with a lab instructor and only **one student** must complete the lab submission as explained in the lab assignment on Canvas.
- Write your lab report following the **report template** provided on Canvas. If you are working with a partner, write both names/IDs on the cover page. It is **prohibited** to include a partner's name on a lab report cover page if that partner does not actively participate in implementing the lab.
- The pre-lab questions prepare you for the challenges you will be presented with in the actual lab. Have their answers ready before the lab session and include these answers in the lab report together with the lab assignments.
- For each lab, submit the following on Canvas before the announced due date/time (only submissions on Canvas are accepted):
    - The lab report as a single Word or PDF document
    - Any files the assignments might ask for (e.g., a demonstration video file)
    - Submit each of the above files separately (do not upload compressed files).
- You can submit multiple attempts for this lab; however, only what you submit in the last attempt will be graded (i.e., all required reports and files must be included in this last attempt).
- **Late submission** of any lab assignment after the given due date and time will be penalized at 1 point per every late hour. No assignment submission will be accepted 48 hours after its due date/time.
- **Successful submission**: it is your responsibility to verify that you submitted successfully all the required files for the lab. First, to verify that your files were submitted, go to the assignment details page, and make sure that it says "Successfully submitted" under grade. Second, you need to validate your submission details by clicking the Submission and Rubric link on that page.
- **Deliberate plagiarism** in any assignment is an extremely serious offense that may result in failing the course. Plagiarism includes submitting answers that are not yours (e.g., copied from someone else or from the Internet). Copying materials from the course slides posted on Canvas is not considered plagiarism.